

1. What is a Window Function?

A **Window Function** lets you perform a calculation **across a set of rows related to the current row**, without collapsing them into one.

 **Keyword: OVER()**

It uses the `OVER()` clause to define the “window” — the subset of rows that influence the result of each row.

Real-Life Analogy:

Scenario: You're viewing employee records.

- A **GROUP BY** on `department` + `AVG(salary)` → Gives **one row per department**.
- A **Window Function** like `AVG(salary) OVER (PARTITION BY department)` → Gives **one row per employee + their department's avg salary as a new column**.

You keep the full detail **and** gain the group-based calculation.

Key Features:

- Keeps **all original rows**.
 - Adds **calculated data** per row.
 - Operates on a **"window" of rows**, not the whole table blindly.
-

2. Why Use Window Functions?

They're great for:

Use Case	Example
Ranking	Top 3 products per region
Running totals	Cumulative monthly sales
Moving average	7-day moving avg of daily users
Comparisons	Compare this month's sales to previous month (LAG/LEAD)

3. Syntax – The OVER() Clause

```
FUNCTION() OVER (  
  PARTITION BY column  
  ORDER BY column  
  ROWS/RANGE clause  
)
```

Components:

➤ PARTITION BY (Optional)

- Divides data into **logical groups**.
- Like a mini **GROUP BY** only for that function.
- Works independently for each group.

AVG(salary) OVER (PARTITION BY department)

➤ ORDER BY (Important!)

- Orders rows **within each partition**.
- Essential for **ranking, running totals, LAG/LEAD**.

ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)

➤ ROWS / RANGE (Advanced)

- Narrows the frame even further:
 - E.g., just previous row, current row, next row.
- Used in **moving averages**, **cumulative sums**, etc.

SUM(sales) OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)



4. Common Window Functions with Examples

We'll use this **employees** table:

emp_id	name	department	salary
1	Alice	Sales	70000
2	Bob	Sales	80000
3	Charlie	Sales	70000
4	David	IT	90000
5	Eve	IT	95000
6	Frank	HR	60000



A. Aggregate Window Functions

➤ Use case:

Show each employee's salary and their department's average salary:

```
SELECT
  name,
  department,
  salary,
  AVG(salary) OVER (PARTITION BY department) AS avg_dept_salary
FROM employees;
```

✓ **Output:**

name	dept	salary	avg_dept_salary
Frank	HR	60000	60000.00
David	IT	90000	92500.00
Eve	IT	95000	92500.00
Alice	Sales	70000	73333.33
Bob	Sales	80000	73333.33
Charlie	Sales	70000	73333.33

👉 Great for **comparison metrics**: avg, sum, count per group – without collapsing rows.

🚩 **B. Ranking Window Functions**

Function	Behavior
<code>ROW_NUMBER()</code>	Unique number per row (no ties)
<code>RANK()</code>	Same rank for ties, skips numbers
<code>DENSE_RANK()</code>	Same rank for ties, doesn't skip

➤ **Use case:**

Rank employees by salary within each department:

```
SELECT
  name,
  department,
  salary,
```

```

ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS
row_num,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rnk
FROM employees;

```

✓ Output:

name	dept	salary	row_num	rnk	dense_rnk
Bob	Sales	80000	1	1	1
Alice	Sales	70000	2	2	2
Charlie	Sales	70000	3	2	2

👉 **ROW_NUMBER** always unique.

👉 **RANK** skips after tie (2,2, then 4).

👉 **DENSE_RANK** does not skip (2,2, then 3).

🔄 C. Value (Offset) Window Functions

Function	What it does
----------	--------------

LAG(col)	Gets value from previous row
-----------------	------------------------------

LEAD(col)	Gets value from next row
------------------	--------------------------

➤ Use case:

Compare current salary with next highest salary:

```

SELECT
name,
department,
salary,
LEAD(salary, 1, 0) OVER (PARTITION BY department ORDER BY salary DESC) AS
next_highest_salary
FROM employees;

```

✓ Output:

name	dept	salary	next_highest_salary
Eve	IT	95000	90000
David	IT	90000	0
Bob	Sales	80000	70000
Alice	Sales	70000	70000
Charlie	Sales	70000	0
Frank	HR	60000	0

👉 Use this for **row-wise comparisons**, **difference calculations**, or **change tracking**.



5. Summary

Feature	Description
What	Performs row-aware calculations without grouping
How	Uses OVER() clause with optional clauses
Key Clauses	PARTITION BY , ORDER BY , ROWS
Use Cases	Rankings, Running Totals, Comparisons, Aggregates



Final Tips for Mastery:

1. Start with **OVER(PARTITION BY)** for simple analytics.
2. Then learn **ORDER BY** inside **OVER()** for **ranking and comparisons**.
3. Gradually experiment with **ROWS BETWEEN** for **moving totals**.

4. Practice using datasets like employees/sales/orders for real scenarios.
 5. Use in interview scenarios: “Find top 3 products per category”, “find users whose transaction value increased vs last month” etc.
-

✓ 1. Practice Questions for Window Functions

Use this `sales` table:

order_id	user_id	product	amount	order_date
101	1	Phone	20000	2024-01-01
102	1	Charger	1500	2024-01-05
103	2	Laptop	50000	2024-01-03
104	2	Mouse	1000	2024-01-06
105	1	Cover	500	2024-01-10
106	3	Tablet	25000	2024-01-02

💡 **Q1: Show each user's total spend per order and total spend overall.**

sql

CopyEdit

```
SELECT
```

```
  user_id,
```

```
  product,
```

```
  amount,
```



```
SUM(amount) OVER (PARTITION BY user_id) AS total_user_spend  
FROM sales;
```

 **Q2: For each order, show the cumulative spend of that user up to that order (running total).**

sql

CopyEdit

```
SELECT  
  
    user_id,  
  
    order_date,  
  
    amount,  
  
    SUM(amount) OVER (  
  
        PARTITION BY user_id  
  
        ORDER BY order_date  
  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
  
    ) AS running_total  
FROM sales;
```

 **Q3: Find orders where the user spent more than their previous order.**

sql

CopyEdit

```
SELECT *,  
  
    LAG(amount) OVER (PARTITION BY user_id ORDER BY order_date) AS  
    prev_order_amount  
  
FROM sales  
  
WHERE amount > LAG(amount) OVER (PARTITION BY user_id ORDER BY  
order_date);
```



Q4: Show first order date for each user in every row.

sql

CopyEdit

```
SELECT *,  
  
    MIN(order_date) OVER (PARTITION BY user_id) AS first_order_date  
  
FROM sales;
```



2. Visualizing the Window Frame (Very Important)

Let's look at a **running total** with:

sql

CopyEdit

```
SUM(amount) OVER (  
  
    PARTITION BY user_id  
  
    ORDER BY order_date  
  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

)

For user_id = 1:

order_date	amount	Running Total
------------	--------	---------------

Jan 1	20000	20000
-------	-------	-------

Jan 5	1500	21500
-------	------	-------

Jan 10	500	22000
--------	-----	-------

 The "window" starts at the first row and ends at the current row — it **slides forward** as we move down the rows.

3. Advanced Patterns Using Window Functions

A. Moving Average (e.g., 3-day window)

sql

CopyEdit

```
SELECT
```

```
    order_date,
```

```
    amount,
```

```
    AVG(amount) OVER (
```

```
        ORDER BY order_date
```

```
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
    ) AS moving_avg  
FROM sales;
```

🧠 Calculates average of current + 2 previous orders = Smooths out fluctuations.

⚠️ B. Detect Gaps in Dates (using LAG)

sql

CopyEdit

```
SELECT *,  
  
    DATEDIFF(  
  
        order_date,  
  
        LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date)  
  
    ) AS days_between_orders  
FROM sales;
```

👁️ Useful for churn detection, checking delayed activity, etc.

🚩 C. Flag First Order

sql

CopyEdit

```
SELECT *,  
  
    CASE  
  
        WHEN ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date)  
        = 1 THEN 'Yes'
```

```
ELSE 'No'  
  
END AS is_first_order  
  
FROM sales;
```

📌 Common in customer journey analysis: first order, first purchase in category, etc.

Where Are These Used in Real Life?

Task	Window Use
Product Analytics	Rank top 3 products per category
Retention Analysis	Compare day-0 vs day-7 activity using LAG
Finance Reports	Cumulative spend / moving averages
Anomaly Detection	Flag users whose spending suddenly drops
Marketing Funnels	Track user journey by stage using RANK

Bonus Tip – Combine with CTEs!

sql

CopyEdit

```
WITH ranked_orders AS (
```

```
SELECT *,  
       RANK() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rnk  
FROM sales  
)  
SELECT * FROM ranked_orders WHERE rnk = 1;
```

👉 Helps **layer logic**, e.g., “Get the highest order per user”.